HILL CLIMBING N-QUEENS

```
import random


def cost(state):
    # number of pairs of queens attacking each other
    h = 0
    n = len(state)
    for i in range(n):
        for j in range(i + 1, n):
            if state[i] == state[j] or abs(state[i] - state[j]) == j - i:
                h += 1
    return h


def print_board(state):
    n = len(state)
    for r in range(n):
        print(" ".join("Q" if state[c] == r else "." for c in range(n)))
    print()


def hill_climbing(state):
    while True:
        h = cost(state)
        if h == 0:
            return state                    # goal reached
        best_state, best_cost = None, h
        for col in range(len(state)):
            for row in range(len(state)):
                if row != state[col]:
```

```
            nb = state[:]
            nb[col] = row           # move one queen within its column
            c = cost(nb)
            if c < best_cost:
                best_state, best_cost = nb, c
    if best_state is None:              # no better neighbor
        return None
    state = best_state

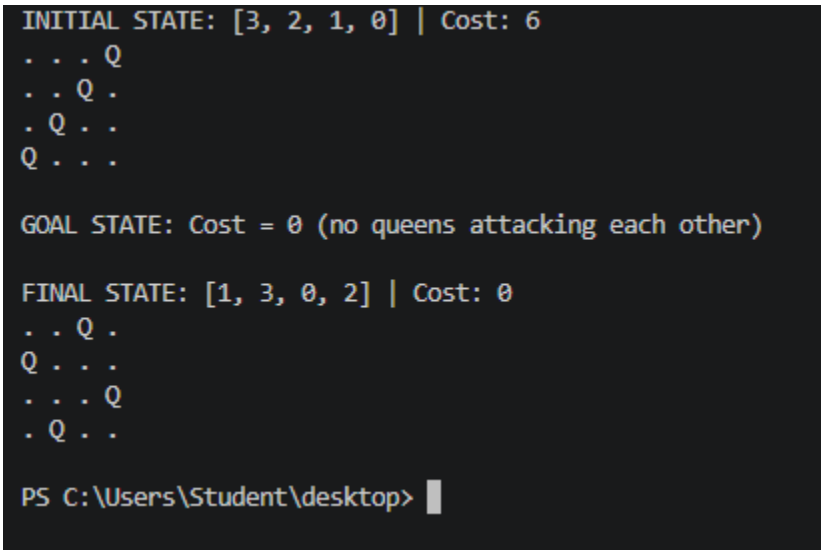
n = 4
initial = [3, 2, 1, 0]                  # index = column, value = row

print("INITIAL STATE:", initial, "| Cost:", cost(initial))
print_board(initial)

result = hill_climbing(initial)
while result is None:                   # stuck -> random restart
    initial = [random.randint(0, n - 1) for _ in range(n)]
    result = hill_climbing(initial)

print("GOAL STATE: Cost = 0 (no queens attacking each other)\n")
print("FINAL STATE:", result, "| Cost:", cost(result))
print_board(result)
```

OUTPUT :


INITIAL STATE: [3, 2, 1, 0] | Cost: 6
. . . Q
. . Q .
. Q . .
Q . . .
GOAL STATE: Cost = 0 (no queens attacking each other)
FINAL STATE: [1, 3, 0, 2] | Cost: 0
. . Q .
Q . . .
. . . Q
. Q . .
PS C:\Users\Student\desktop>